

Golden Coffee: Assignment 1

Authors: Mukhitden Nurdaulet, Arlan Kudabay.

Project: Golden Coffee, Astana (65/2 Uly Dala Avenue).

Task A: Anatomy of a real page

Website analyzed: Coffee Boom — <https://coffeeboom.kz/en/>.

- Doctype: `<!DOCTYPE html>; language: lang="en"`.
- Encoding: `meta http-equiv="Content-Type" content="text/html; charset=utf-8"`.
- Viewport: `width=device-width, initial-scale=1.0`.
- Title: Coffee Boom - a coffee company in Kazakstan, Russia and Uzbekistan.

Three additional meta tags:

1. `http-equiv="x-ua-compatible" content="IE=edge"`
2. `name="Keywords" content=""`
3. `name="theme-color" content="whitesmoke"`

The Coffee Boom page source contains header, main and footer, one h1 and two h2 elements. It lacks semantic tags such as `nav`, `section`, `article` or `aside`, using generic `div` containers instead. It does not have tables or form structures for ordering.

Three structural issues and how we avoided them:

1. The site uses `div.h_menu` for navigation without a `nav` element or ARIA landmark. In our six pages, we enclosed all navigation links inside a semantic `nav` tag with an accessible `aria-label="Main navigation"`.
2. Headings on the site (such as "Formats") are styled inside `div.l-text` instead of heading tags. Across our project (`menu.html` and `drinks.html`), we used `h2` for major content sections and `h3` inside article cards for individual dish and drink descriptions.
3. The cafe search bar has only a placeholder with no linked label (source line 2060). In `visit.html` and `feedback.html`, every form field has an explicit label linked via the `for` and `id` attributes.

Task B: Written explanation (170 words)

Our web pages are pure HTML documents containing structured text, semantic containers and relative links. The browser reads our HTML files as UTF-8 encoded text and parses the markup into a hierarchical Document Object Model (DOM) tree. It calculates the geometry and visual placement of each box based on user-agent default styles and renders the layout on the screen without any external CSS or JavaScript.

In `index.html`, `main` separates the core introductory content from the repeated contact address in the footer. In `menu.html` and `drinks.html`, each article contains a self-contained breakfast or drink item that can be read independently outside the section context. In `feedback.html`, `nav` defines the primary website navigation bar. A `div` tag would fail to communicate the machine-readable role of these blocks. We strictly restricted `div` elements to non-semantic groupings, such as grouping submit and reset buttons inside forms.

When a user clicks Submit on `visit.html` or `feedback.html`, the browser performs client-side validation against required fields, email syntax and min values. When valid, the browser executes an HTTP GET request that appends field values to the current URL in the navigation bar. Because there is currently no backend server script handling requests, the

local HTML document simply refreshes and re-displays itself with query parameters in the address.